A Controlled, End-to-End Comparison of EC2, ECS Fargate, and AWS Lambda

for Stateless Web Services

Ma Liyu Zheng Junlan Zheng Zelan

CS5296 Cloud Computing, Spring 2026

City University of Hong Kong, Group 24

April 2026

github.com/Fighterforever/cs5296

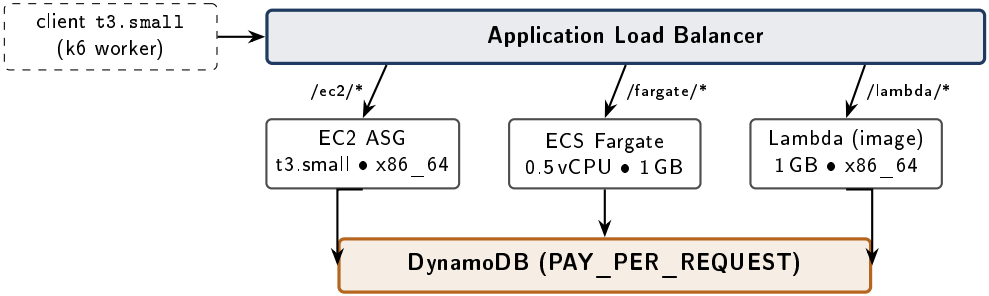
Three ways to run the same web service on AWS

EC2	ECS Fargate	AWS Lambda
Virtual machines.	Managed containers.	Functions on demand.
Predictable performance.	AWS handles the host.	No servers, no idle cost.
You handle scaling and OS upkeep.	*Your* task definition decides the size.	Cold-start penalty after idle.
Billed by the running hour.	Billed by the vCPU-second.	Billed per request + per GB-second.

Performance, elasticity, and cost rarely move in the same direction.

Most public studies compare different applications. We compare them apples to apples.

One container image, three runtimes, one ALB



- Same OCI image (`shortlink:v2`) deployed unmodified to all three.
- Lambda runs the same image via the AWS Lambda Web Adapter, translating invocations into HTTP calls to localhost.
- Shared backend (DynamoDB) and shared frontend (ALB) confine measurable differences to the compute layer.

Four scenarios, driven by the same k6 harness

Steady-state RPS sweep

Constant arrival rate at four levels (50/100/200/400 RPS for EC2 and Fargate; 25/50/75 for Lambda). Recovers per-stage tail latency.

Step-arrival burst

One minute idle, three minutes at the target rate, one minute idle. Captures elasticity and time-to-steady-state.

Cold-start probe

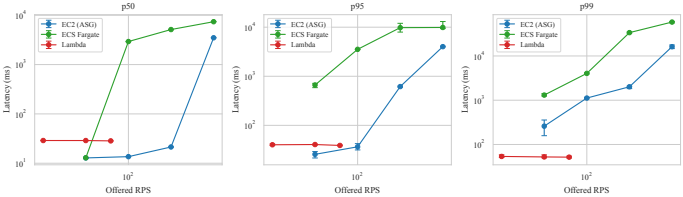
Twelve sparse `/healthz` requests with 60-second gaps, designed to characterise warm-vs-cold latency on Lambda.

Mixed read/write

70 % GET-redirect, 30 % POST-shorten at 200 RPS for three minutes. Approximates real shortener traffic.

All scenarios driven from a co-located `t3.small` k6 worker in `us-east-1a`; raw timings recorded in CSV;

Steady-state: Lambda is flat, EC2 has a fat tail, Fargate saturates

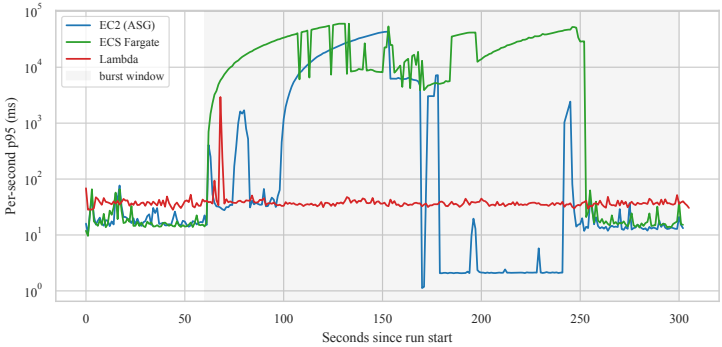


Lambda (1 GB)
p50 ≈ 29 ms, p95 ≈ 40 ms, p99 ≈ 52 ms across the entire band.

EC2 (t3.small ASG)
Tight 13 ms median up to 200 RPS. p95 goes from 26 ms to over 600 ms as the burstable-credit budget runs out.

Fargate (0.5 vCPU)
Saturates by 100 RPS. At 400 RPS, 6.9 % of requests hit the ALB idle timeout.

Elasticity: per-second p95 through a 3-minute burst

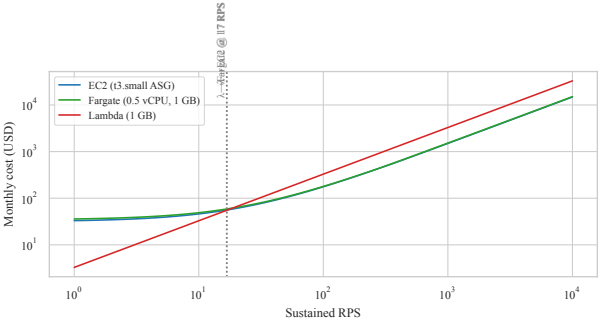


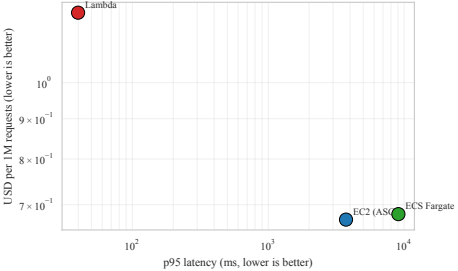
Lambda — tracks its steady envelope from the first second.

EC2 — takes about a minute for the second ASG instance to come online and rejoin the load balancer.

Fargate — the single task collapses; ECS service autoscaling at the default 30-second cooldown is too slow to recover.

Cost: a single break-even crossover near 20 RPS





- Below ∼20 RPS, Lambda's no-idle billing wins because EC2 and Fargate pay around the clock.
- Above ∼20 RPS, a single provisioned instance amortises across enough requests that the flat hourly rate beats per-invocation pricing.
- Pareto chart at 50 RPS: Lambda is strictly dominant in our measured regime.

Live demo

Deployment, benchmark harness, and reproduction.

Reproducibility in five `make` targets

One repo, one command per phase

```
git clone https://github.com/Fighterforever/cs5296.git && cd cs5296
```

`make image`	build the OCI container image (`linux/amd64`, multi-runtime)
`make up`	`tofu apply`: 39 AWS resources in ∼7 minutes
`make bench`	drive the 4-scenario k6 matrix against the live ALB
`make analyze`	bootstrap stats, write 5 figures and 4 CSV summaries
`make report`	compile the LaTeX report end-to-end
`make nuke`	destroy + sweep stray ENI/EIP

- Infrastructure-as-code in OpenTofu/Terraform; portable to any AWS account.
- Analysis pipeline in Python; figures regenerate from raw k6 CSV in under a minute.
- Total measured AWS spend for one full matrix run: $< \$1$.

Three findings, two honest threats to validity

Findings

1. Lambda dominates up to ∼20 RPS — no-idle billing plus the most stable tail in the measured band.
2. EC2 keeps a tight median further than expected (200 RPS), but its tail blows out once the burstable-credit budget is depleted.
3. Fargate at the smallest size is *not* Lambda-with-containers — a single 0.5 vCPU task cannot absorb 100 RPS of Python.

Threats to validity

- Cold-start probe used 60-second gaps, too short to reliably reclaim Lambda containers (typical reclaim 5–15 min). All probe samples landed in the warm band; one genuine cold start (4.3 s) captured incidentally.
- New AWS account default Lambda concurrency was 10; this capped our Lambda peak at 75 RPS. A quota-increase ticket would lift it.

Thank you

Code, raw measurements, figures, and the report are public:

`github.com/Fighterforever/cs5296`